

Stream API

Stream

- ❖ **Stream** — Java 8'da kiritilgan *java.util.stream* paketidagi interfeys bo'lib, u ma'lumotlar ketma-ketligini (sequence of elements) qayta ishlash uchun ishlatiladi. U kolleksiyalar, massivlar yoki boshqa ma'lumot manbalari bilan ishlashni deklarativ va funktsional usulda amalga oshirish imkonini beradi. Stream o'zi funktsional interfeys emas (chunki unda bir nechta abstrakt metodlar mavjud), lekin u funktsional dasturlash tamoyillariga asoslanadi va Behavior Parameterization-ni qo'llab-quvvatlash uchun *Predicate*, *Function*, *Consumer*, *Supplier* kabi funktsional interfeyslar bilan yaqindan ishlaydi.
- ❖ **Asosiy xususiyat:** Ma'lumotlarni bir marta qayta ishlash (bir yo'nalishli oqim)
- ❖ **Maqsad:** Ma'lumotlarni filtrlashtirish, transformatsiya qilish, saralash, yig'ish va boshqa operatsiyalarni samarali bajarish.

Declarative va Imperative Programming

- ◆ **Declarative programming** — bu dasturlash paradigmasi bo'lib, unda dasturchi kompyuterga *nima qilish kerakligini* aniq ko'rsatmalar (qadamlar) bilan emas, balki *qanday natija olish kerakligini* tasvirlash orqali bildiradi. Bu paradigma **imperative programming**'dan farq qiladi, chunki imperativ dasturlashda dasturchi har bir qadamni aniq belgilab, algoritmni qo'lda yozadi.

Streamning asosiy xususiyatlari

1. **Deklarativlik:** Kod “nima qilish kerak”ligini tasvirlaydi, “qanday qilish” jarayoni esa ichki optimizatsiyaga bog‘liq.
2. **Laziness (kechiktirilgan bajarish):** Operatsiyalar faqat terminal operatsiyasi (masalan, collect, forEach) chaqirilganda bajariladi.
3. **Bir martalik:** Har bir Stream faqat bir marta ishlatiladi; qayta ishlatish uchun yangi Stream yaratish kerak.
4. **Parallel ishlash:** parallelStream() yordamida ko‘p oqimli (multithreaded) qayta ishlashni qo‘llab-quvvatlaydi.
5. **Funksional interfeyslar bilan integratsiya:** Ko‘p metodlari funksional interfeys parametrlarni qabul qiladi.

Streamning asosiy metodlari

Stream operatsiyalari ikkiga bo'linadi: **intermediate** (oralik) va **terminal** (yakuniy).

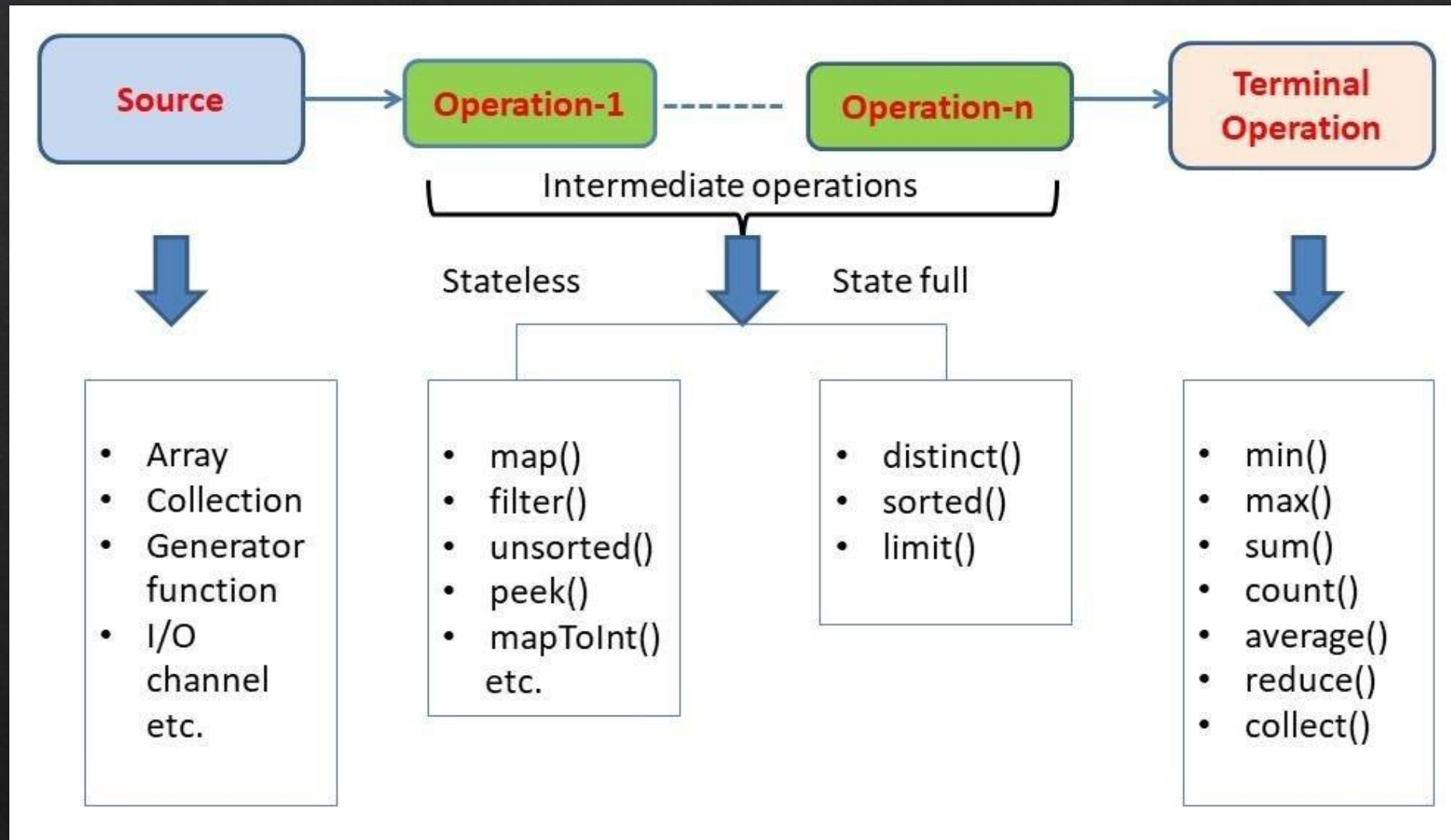
Intermediate metodlar (yangi Stream qaytaradi):

- **filter(Predicate<T> predicate):** Elementlarni shart asosida filtrlaydi.
- **map(Function<T, R> mapper):** Elementlarni transformatsiya qiladi.
- **sorted(Comparator<T> comparator):** Elementlarni saralaydi.
- **distinct():** Takrorlanuvchi elementlarni olib tashlaydi.
- **limit(long maxSize):** Elementlar sonini cheklaydi.

Terminal metodlar (natija qaytaradi yoki Streamni yakunlaydi):

- **forEach(Consumer<T> action):** Har bir elementga amal bajaradi.
- **collect(Collector<T, A, R> collector):** Elementlarni kolleksiyaga yig'adi.
- **count():** Elementlar sonini qaytaradi.
- **reduce(BinaryOperator<T> accumulator):** Elementlarni birlashtiradi.
- **anyMatch(Predicate<T> predicate):** Kamida bitta element shartga mos kelishini tekshiradi.

- **Intermediate operation** – yangi stream yaratib beradi;
- **Terminal operation** – bir nechta streamlarni tugatuvchi operation hisoblanadi.



- ❖ **Stateless operatsiyalar** — bu Stream'dagi har bir elementni qayta ishlashda boshqa elementlardan mustaqil ravishda ishlaydigan operatsiyalar. Ular o'z holatini (state) saqlamaydi va har bir elementni alohida-alohida qayta ishlaydi.
- ❖ **Stateful operatsiyalar** — bu Stream'dagi elementlarni qayta ishlashda boshqa elementlar haqida ma'lumot (holat) saqlashni talab qiladigan operatsiyalar. Ular natija hosil qilish uchun butun oqim yoki uning bir qismiga bog'liq bo'ladi.

Stream vs Collection

- ◆ **Collection** — Java'dagi ma'lumotlar to'plamini (data structure) saqlash va boshqarish uchun ishlatiladigan interfeys (`java.util.Collection`).
- ◆ **Stream** — Java 8'da kiritilgan `java.util.stream` paketidagi interfeys bo'lib, ma'lumotlar oqimini (data pipeline) qayta ishlash uchun ishlatiladi. U ma'lumotlarni saqlamaydi, balki ularga operatsiyalarni ketma-ket qo'llab, natija hosil qiladi.

Collection va Stream o'rtasidagi asosiy farqlar

Xususiyat	Collection	Stream
Maqsad	Ma'lumotlarni saqlash va boshqarish	Ma'lumotlarni qayta ishlash
Ma'lumot saqlash	Ha, xotirada saqlanadi	Yo'q, faqat oqim sifatida ishlaydi
Qayta ishlatish	Har doim qayta ishlatiladi	Bir marta ishlatiladi (single-use)
Operatsiya usuli	Imperativ (tsikl bilan)	Deklarativ (funktsional usulda)
Parallelizm	Qo'lda boshqariladi	Oson parallel qilinadi (parallelStream)
Elementlar soni	Aniqlanadi (size())	Aniqlanmasligi mumkin (infinite bo'lishi mumkin)
Ishlash vaqti	Elementlar darhol mavjud	Lazy evaluation (kerak bo'lganda bajariladi)

Stream sources

- ❖ **Stream source** — bu Java'dagi Stream ob'ektini yaratish uchun ishlatiladigan ma'lumotlar manbai.

- ❖ Java'da Stream'ni hosil qilish uchun quyidagi manbalardan foydalanish mumkin:
 1. **Collection**'dan Stream
 2. Array'dan Stream; **Arrays.stream()**, Primitive types uchun - **IntStream**, **LongStream**, **DoubleStream**.
 3. **Stream.of()** metodi
 4. **Stream.iterate()** bilan cheksiz Stream
 5. **Files** class'i (*java.nio.file*) fayl mazmunidan Stream hosil qilish uchun metodlar taqdim etadi; **Files.lines()**, **Files.list()**.
 6. **BufferedReader.lines()**
 7. **Rando.ints()**

Stream Truncation

- ◆ **Stream Truncation**— bu Stream oqimidagi elementlar sonini kamaytirish yoki faqat kerakli qismini tanlab olish jarayoni. Bu operatsiya odatda cheksiz (infinite) Stream'lar bilan ishlashda yoki katta hajmdagi ma'lumotlardan faqat bir qismini qayta ishlashda ishlatiladi. Java'da bu uchun asosiy metodlar: **limit()** va **skip()**.
- ◆ 1. **limit(long maxSize)** - Stream'dan faqat birinchi maxSize ta elementni oladi va qolganlarini chetlashtiradi.
- ◆ 2. **skip(long n)** - Stream'dan birinchi n ta elementni o'tkazib yuboradi va qolganlarini qaytaradi.

Consuming Stream

- ◆ **Consuming Stream** — bu Stream oqimida mavjud elementlarni yakuniy natija sifatida ishlatish yoki qayta ishlash jarayonidir. Stream'lar **lazy evaluation** (*kerak bo'lganda bajarish*) prinsipi asosida ishlaydi, ya'ni intermediate (oraliq) operatsiyalar faqat terminal (yakuniy) operatsiya chaqirilganda bajariladi. Terminal operatsiya Stream'ni "iste'mol qiladi" va undan keyin Stream qayta ishlatib bo'lmaydi. **peek()** va **forEach()**.
- ◆ **peek** — bu Streamning **intermediate (oraliq)** operatsiyasi bo'lib, har bir elementga Consumer orqali amal qo'llash imkonini beradi, lekin Stream'ni o'zgartirmaydi va oqimni davom ettiradi. U asosan debugging (xatolarni aniqlash) yoki oqim elementlarini tekshirish uchun ishlatiladi. Intermediate (Stateless).
- ◆ **forEach** — bu Streamning **terminal (yakuniy)** operatsiyasi bo'lib, har bir elementga Consumer orqali amal qo'llaydi va Stream'ni iste'mol qilib, yopadi. U natija qaytarmaydi va keyingi operatsiyalar bilan zanjirlanmaydi. Terminal (Stateless).

Matching

- ◆ **Matching operatsiyalari** — bu Stream elementlarini berilgan shart (**Predicate**) asosida tekshirib, boolean natija qaytaradigan terminal operatsiyalar. Ular Stream'dagi elementlarning hammasi, birortasi yoki hech biri shartga mos kelishini aniqlash uchun ishlatiladi.
- ◆ Java'da uchta asosiy matching operatsiyasi mavjud:
- ◆ 1. **anyMatch(Predicate<T>)** - Stream'dagi kamida bitta element shartga mos kelsa, true qaytaradi.
- ◆ 2. **allMatch(Predicate<T>)** - Stream'dagi barcha elementlar shartga mos kelsa, true qaytaradi.
- ◆ 3. **noneMatch(Predicate<T>)** - Stream'dagi hech bir element shartga mos kelmasa, true qaytaradi.

Bo'sh Stream: Bo'sh Stream'da:

- **anyMatch:** false qaytaradi (hech qanday moslik yo'q).
- **allMatch:** true qaytaradi (hech qanday nomoslik yo'q).
- **noneMatch:** true qaytaradi (hech qanday moslik yo'q).

Mapping

- ◆ **Mapping Operations** - bu Streamdagi har bir elementga funksiya qo'llab, uni yangi qiymatga aylantiradigan intermediate (oraliq) operatsiyalar. Ular Stream'ni o'zgartirmaydi, balki yangi Stream hosil qiladi, unda elementlar transformatsiya qilingan shaklda bo'ladi.
- ◆ `map(Function<T, R>)`
- ◆ `mapToInt(ToIntFunction<T>)`, `mapToLong(ToLongFunction<T>)`,
`mapToDouble(ToDoubleFunction<T>)`
- ◆ `flatMap(Function<T, Stream<R>>)`
- ◆ `flatMapToInt`, `flatMapToLong`, `flatMapToDouble`

Finding element

- ◆ **Finding Operations** - bu Streamdagi elementlar orasidan ma'lum bir elementni topib, uni Optional shaklida qaytaradigan terminal operatsiyalar. Ular odatda birinchi yoki ixtiyoriy elementni topish uchun ishlatiladi.
- ◆ **findFirst()** - Stream'dagi birinchi elementni qaytaradi (tartib bo'yicha)
- ◆ **findAny()** - Stream'dagi ixtiyoriy (har qanday) elementni qaytaradi. (Parallel Streaming)

Stream Reduction

- ◆ **Reduction (qisqartirish)** — bu Streamdagi barcha elementlarni ma'lum bir qoidaga asosan birlashtirib, yakuniy bitta natija (yoki umumiy qiymat) hosil qilish jarayoni. Bu operatsiya terminal operatsiya sifatida Stream'ni iste'mol qiladi va ko'pincha yig'indi, ko'paytma, eng katta/kichik qiymat yoki boshqa umumiy natijani hisoblash uchun ishlatiladi.
- ◆ Java'da Stream reduction asosan **reduce** metodi orqali amalga oshiriladi, lekin uning bir nechta shakllari mavjud:
- ◆ **reduce(BinaryOperator<T>)** - Elementlarni *BinaryOperator* yordamida birlashtiradi, boshlang'ich qiymat yo'q.
- ◆ **reduce(T identity, BinaryOperator<T>)** - Boshlang'ich qiymat (identity) bilan elementlarni birlashtiradi.
- ◆ **reduce(U identity, BiFunction<U, ? super T, U> accumulator, BinaryOperator<U> combiner)** - Boshlang'ich qiymat, accumulator (yig'uvchi) va combiner (birlashtiruvchi) bilan murakkab reduction.

toArray

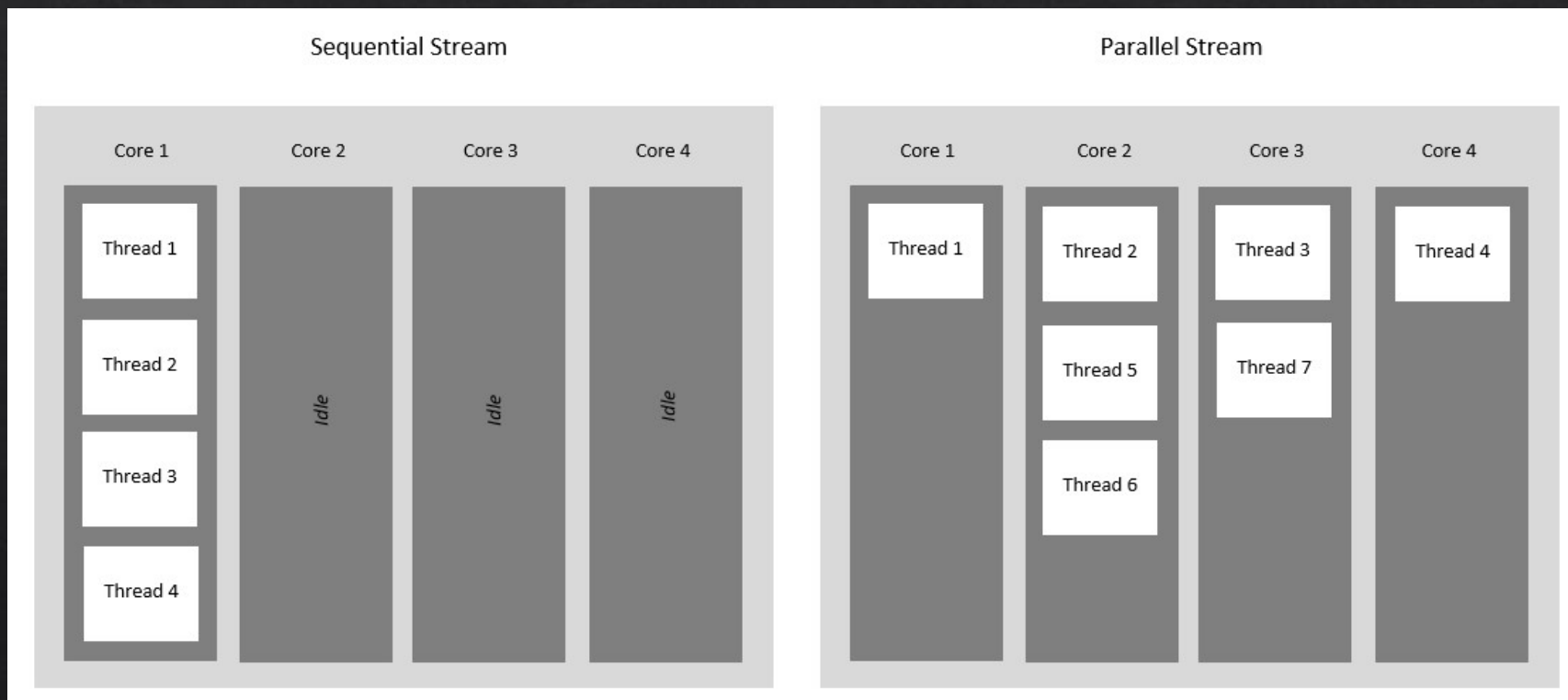
- ◆ **toArray** — bu Streamdagi barcha elementlarni massiv (array) shaklida qaytaradigan terminal operatsiya. U Stream'ni iste'mol qilib, elementlarni saqlash uchun qulay va an'anaviy ma'lumot tuzilmasi — massivga o'tkazadi.

Infinite Streams

- ◆ **Infinite Streams** — bu elementlari cheklanmagan, ya'ni cheksiz ketma-ketlik hosil qila oladigan Stream'lar. Ular Java'da ma'lumotlarning cheksiz oqimini ifodalash uchun ishlatiladi va odatda matematik ketma-ketliklar, tasodifiy sonlar yoki doimiy generatsiya qilinadigan qiymatlarni modellashtirish uchun qo'llaniladi.
- ◆ 1. **Stream.iterate()** - Boshlang'ich qiymatdan boshlab, har bir keyingi elementni funksiya yordamida hosil qiladi.
- ◆ 2. **Stream.generate()** - Har bir elementni Supplier yordamida generatsiya qiladi.
- ◆ 3. **Random.ints(), Random.longs(), Random.doubles()**: Cheksiz tasodifiy primitiv Stream'lar.
- ◆ (takeWhile)

Parallel Stream

- ◆ **Parallel Stream** — Java 8'da kiritilgan Streamning ko'p oqimli versiyasi bo'lib, ma'lumotlarni qayta ishlashni bir nechta thread'larda parallel ravishda bajarish imkonini beradi. U `java.util.stream` paketida joylashgan va **`parallelStream()`** yoki **`stream().parallel()`** metodlari orqali yaratiladi. Parallel Stream'lar *Fork/Join Framework* asosida ishlaydi va umumiy *Thread Pool*'dan (odatda **`ForkJoinPool.commonPool()`**) foydalanadi.



Thank you for learning with us.

Developer: Tokhir Asadov (Java Software Engineer)

Company: pdp.uz | javatuz.com

Email: guvalakat1603@gmail.com

Telegram: [@Asadov_Tokhir](https://t.me/Asadov_Tokhir)

